

Programmation Orientée Objet pour le Calcul Scientifique : Part II Deep Water

Organisation du cours

Sous forme d'**Item**, se raccrochant à trois catégories :

- Outils et Langage (Tool)
- Gestion de la mémoire (Mem)
- Orienté objet (OO)
- **Programmation statique (Static)**

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...

Item 20 Encore une référence...

Nous avons vu précédemment `&&`, référence d'une *rvalue* (ou objet temporaire)

- En programmation statique `&&` peut prendre un autre sens, un peu plus général
- Lorsqu'il apparaît dans la signature d'une fonction/méthode template

```
template <typename T>  
void func(T&& param) {} // universal reference
```

- Dans ce cas il signifie que l'objet passé peut-être soit :
 - de type `T&`
 - de type `T&&`

On peut les appelle **forwarding reference** (*référence de transfert*)
car elles permettent le **perfect forwarding**

Item 20 Encore une référence...

Les *forwarding reference* peuvent aussi permettre une déclaration : `auto &&`

Comme dans le cas template, Le type déduit pour l'objet peut être

- une référence de *lvalue* `type&`
- une référence de *rvalue* `type&&`

```
auto && value = 1; // value is deduced to be of type int&&  
int i = 0;  
auto && value2 = i; // value2 is deduced to be of type int&
```

Le type déduit est au plus près du type en originel. Pour rappel :

```
auto value = 1; // value is deduced to be of type int  
auto value2 = i; // value2 is deduced to be of type int
```

Item 20 Encore une référence...

Quel(s) intérêt(s) ?

- une signature unique pour traiter à la fois des références de *rvalue* ou de *lvalue*
- une déclaration unique pour stocker à la fois des références de *rvalue* ou de *lvalue*

Item 20 Encore une référence...

Une signature unique pour traiter à la fois des références de *rvalue* ou de *lvalue*

```
template <typename T>
void func(T&& param) {
    std::cout << param << std::endl;
} // forwarding reference

int main(){
    int i = 0;
    func(i); // Call with lvalue. T is deduced to be int&.
    func(5); // Call with rvalue. T is deduced to be int&&
}
```

Remarque : dans l'exemple ci dessus, dans la signature de `func` :

- si `T&&` était remplacé par `T&` l'appel avec une *rvalue* ne serait pas possible.
- Si `T&&` était remplacé par `int&&` l'appel avec une *lvalue* ne serait pas possible .

Item 20 Encore une référence...

Une déclaration unique pour stocker à la fois des références de *rvalue* ou de *lvalue*

```
template <typename T>
void func(T&& param) {
    auto && copy = param; // not yet correct. missing forward
} // universal reference

int main(){
    int i = 0;
    func(i); // Call with lvalue. T is deduced to be int&.
    func(5); // Call with rvalue. T is deduced to be int&&
}
```

Item 20 Encore une référence...

Comment gérer un move conditionnel ?

- Si `T&&` = une référence de *rvalue* on souhaite un déplacement des données `move`
- Si `T&&` = une référence de *lvalue* on souhaite une copie des données
- Pour cela on utilise `std::forward<T>` (header `<utility>`)

```
template <typename T>
void func(T&& param) {
    auto && copy = std::forward<T>(param); } // universal reference

int main(){
    int i = 0;
    func(i); // Call with lvalue. T is deduced to be int&.
    func(5); // Call with rvalue. T is deduced to be int&&
}
```

Item 20 Encore une référence...

Comment gérer un move conditionnel ? Un exemple 1/2

```
void f(int&) {  
    std::cout << "f(int&)" << std::endl;  
}  
void f(int&&) {  
    std::cout << "f(int&& )" << std::endl;  
}  
int main() { // code not yet working, type is to be explicited  
    auto && rvalue_ref = 5;  
    f(std::forward<type>(rvalue_ref)); // type to be explicited in next slide  
    int i = 42;  
    auto&& lvalue_ref = i;  
    f(std::forward<type>(lvalue_ref)); // type to be explicited in next slide  
}
```

Item 20 Encore une référence...

Comment gérer un move conditionnel ? Un exemple 2/2

```
void f(int&) {  
    std::cout << "f(int&)" << std::endl;  
}  
void f(int&&) {  
    std::cout << "f(int&& )" << std::endl;  
}  
int main() { // type is now explicited  
    auto && rvalue_ref = 5;  
    f(std::forward<decltype(rvalue_ref)>(rvalue_ref));  
    int i = 42;  
    auto&& lvalue_ref = i;  
    f(std::forward<decltype(lvalue_ref)>(lvalue_ref));  
}
```

Item 20 Encore une référence...

Quel intérêt ? Un exemple 1/2 ; sans : deux écritures de `copy`

```
struct MyVector{
    MyVector& operator= (MyVector &) {std::cout << "copy data " << std::endl; return *this;}
    MyVector& operator= (MyVector&&) {std::cout << "move data" << std::endl; return *this;}
    void copy(MyVector& vec) {std::cout << "Copy data " << std::endl; *this = vec;}
    void copy(MyVector&&) {std::cout << "Move data " << std::endl; *this = std::move(vec);}
};

int main(){
    MyVector a{},b{};
    a.copy(b);
    a.copy(MyVector{});
}
```

Item 20 Encore une référence...

Quel intérêt ? Un exemple 2/2 ; avec : une unique écriture de `copy`

```
struct MyVector{
private:
    MyVector& operator= (MyVector const&) {std::cout << "copy data " << std::endl; return *this;}
    MyVector& operator= (MyVector&&) {std::cout << "move data" << std::endl; return *this;}
public:
    template <typename T>
    void copy(T&& vec) {std::cout << "copy or move" << std::endl; *this = std::forward<T>(vec);}
};

int main(){
    MyVector a{},b{};
    a.copy(b);
    a.copy(MyVector{});
}
```

Item 20 Encore une référence...

Quel intérêt ? Un deuxième exemple 1/2 ; sans : deux écritures de `addData`

```
struct HeavyData{
    HeavyData& operator=(HeavyData const&) {std::cout << "copy data" << std::endl;return *this;}
    HeavyData& operator=(HeavyData &&) {std::cout << "move data" << std::endl;return *this;}
};
struct Container {
    HeavyData m_data;
    void addData(HeavyData const& heavy_data) {m_data = heavy_data;}
    void addData(HeavyData && heavy_data) {m_data = std::move(heavy_data);}
};

int main() {
    HeavyData a{};
    Container c{};
    c.addData(a); // add lvalue reference by copy
    c.addData(HeavyData{}); // add rvalue reference by move
}
```

Item 20 Encore une référence...

Quel intérêt ? Un deuxième exemple 2/2 ; avec : une unique écriture de `addData`

```
struct HeavyData{
    HeavyData& operator=(HeavyData const&) {std::cout << "copy data" << std::endl;return *this;}
    HeavyData& operator=(HeavyData &&) {std::cout << "move data" << std::endl;return *this;}
};
struct Container {
    HeavyData m_data;
    template <typename T>
    void addData(T&& heavy_data) { m_data = std::forward<T>(heavy_data);}
};

int main() {
    HeavyData a{};
    Container c{};
    c.addData(a); // add lvalue reference by copy
    c.addData(HeavyData{}); // add rvalue reference by move
}
```

Item 20 Encore une référence...

Quel intérêt ? *pour aller plus loin* : vérifier les types avec `if constexpr`

```
struct HeavyData{
    HeavyData& operator=(HeavyData const&) {std::cout << "copy data" << std::endl;return *this;}
    HeavyData& operator=(HeavyData &&) {std::cout << "move data" << std::endl;return *this;}
};
template <typename Data>
struct Container {
    Data m_data;
    template <typename T>
    void addData(T&& data) {
        if constexpr (std::is_same_v<Data, std::remove_reference_t<T>>) m_data = std::forward<T>(data);
        else std::cout << "Bad type added";
    }
};
int main() {
    HeavyData a{};
    Container<HeavyData> c{};
    c.addData(a);
    c.addData(HeavyData{});
    c.addData(42); // "Bad type added"
}
```

Item 20 Encore une référence...

Exo 20.1 : Créez une classe `B` contenant un objet d'une classe `A` comme donnée membre. La classe `B` sera construite à partir d'une *forwarding reference* vers `A` pour pouvoir faire un déplacement mémoire lorsque c'est possible ; une copie sinon. Faites fonctionner le code suivant :

```
int main() {  
    A a{};  
    B b(A{}); // A(A&&)  
    B b2(a); // A(A const&)  
}
```

de sorte qu'il affiche :

```
A(A&&)  
A(A const&)
```

Item 21 Template : mettez le pack

En C++11 a été ajouté la possibilité d'avoir un nombre variable de paramètres template

```
template<typename... T>
void func(T&&... t){}

int main() {
    func(1);
    func(1, 'c', 3.f);
}
```

Item 21 Template : mettez le pack

En C++11 a été ajouté la possibilité d'avoir un nombre variable de paramètre template

- il est tout d'abord possible de connaître le **nombre d'arguments** passés

```
template<typename... T>
void func(T&&...){
    std::cout << " nb parameter : " << sizeof...(T) << std::endl;
}

int main() {
    func(1);
    func(1, 'c', 3.f);
}
```

Item 21 Template : mettez le pack

Un premier cas d'usage pour les fonctions : l'appel pour tous les paramètres

```
void print() {  
    std::cout << std::endl;  
}  
  
template<typename U,typename... T>  
void print(U&& u, T&&... t){  
    std::cout << u << " ";  
    print(t...);  
}  
  
int main() {  
    print(1);  
    print(1, 'c');  
    print(1, "Hello World", 3.14);  
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

- dans une opération "unaire" : pour un parameter pack `args`
 - `(args op ...)` : unary right folding
`(arg1 op (arg2 op (... ( arg_n-1 op arg_n))))`

```
template<typename... T>
int func(T&&... args) {
    return ( args - ...);
}

int main(){
    func(3,2,1); // (3 - (2 - 1 )) = 2
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

- dans une opération "unaire" : pour un parameter pack `args`
 - `(... op args)` : unary left folding
`((arg1 op arg2) op ...) op arg_n)`

```
template<typename... T>
int func(T&&... args) {
    return (... - args);
}

int main(){
    func(3,2,1); // (3 - 2) - 1 = 0
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

- dans une opération "unaire" : pour un parameter pack `args`
 - `(args op ...)` : unary right folding
`(arg1 op (arg2 op (... ( arg_n-1 op arg_n))))`
 - `(... op args)` : unary left folding
`(( (arg1 op arg2) op ...) op arg_n)`

Exo 21.1 : Réalisez une fonction qui additionne les n arguments passés en entrée :

```
int main(){  
    auto result = 0;  
    result = sum(1,2,3);  
    result = sum(1,2,4,5,6);  
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

- dans une opération "binaire" : pour un parameter pack `args`
 - `(args op ... op init)` : binary right folding
`(arg1 op (... op (arg_n-1 op (arg_n op init))))`

```
template <typename U, typename... T>
void init(U value, T&... args) {
    (args = ... = value); // arg1 = arg2 = ... = value
}

int main(){
    int a, b,c;
    init(42,a,b,c);
    std::cout << " a " << a << " b " << b << " c " << c << std::endl;
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

- dans une opération "binaire" : pour un parameter pack `args`
 - `(init op ... op args)` : binary left folding
`((((init op arg1) op arg2) op ...) op arg_n)`

```
template <typename U, typename... T>
U cumul(U value, T... args) {
    return (value += ... += args); // ((value += val1) += val2) += val3)
}

int main(){
    std::cout << " cumul " << cumul(1,a,b,c);
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

- dans une opération "binaire" : pour un parameter pack `args`
 - `(init op ... op args)` : binary left folding
`((((init op arg1) op arg2) op ...) op arg_n)`

```
// With a type check through a requires clause (using folding !)  
template <typename U, typename... T>  
U cumul(U value, T... args) requires (std::is_same_v<U,T> && ...) {  
    return (value += ... += args); // ((value += val1) += val2) += val3)  
}  
  
int main(){  
    std::cout << " cumul " << cumul(1,a,b,c);  
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

- dans une opération "binaire" : pour un parameter pack `args` et un opérateur `op`
 - `(args op ... op init)` : binary right folding
`(arg1 op (... op (arg_n-1 op (arg_n op init))))`
 - `(init op ... op args)` : binary left folding
`(((((init op arg1) op arg2) op ...) op arg_n)`

Exo 21.2 : refaites une fonction d'affichage de ses `n` arguments d'entrée sans utiliser de récursion. Vous utiliserez un "binary left folding" avec l'opérateur `<<`

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

Pour aller plus loin : dans une expression de type `(args operator ...)`

- `args` peut-être une expression contenant un parameter pack non développé
 - par exemple un appel de fonction
- `(func(pack), ...)` va donc donner par exemple pour trois arguments :
 - `(func(arg1), (func(arg2), func(arg3)))`
 - permet d'appeler une fonction pour tous les arguments sans récursion

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

Pour aller plus loin : `(func(pack), ...)`

- permet de appeler une fonction pour tous les arguments sans récursion

```
template <typename T, typename...Args>
addToVector(std::vector<T>& vec, Args... args) {
    (vec.push_back(args),...); // check on type compatibility is omitted
}

int main(){
    std::vector<int> my_vector;
    addToVector(my_vector,1,2,3);
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

Pour aller plus loin : `(func(pack), ...)`

- permet de appeler une fonction pour tous les arguments sans récursion

```
// version 'universal reference'
template <typename T, typename...Args>
void addToVector(std::vector<T>& vec, Args&&... args) {
    (vec.push_back(std::forward<Args>(args)), ...);
}

int main(){
    std::vector<int> my_vector;
    int a = 2;
    addToVector(my_vector, 1, a, 3); // (std::vector<int>&, int&&, int&, int&&)
}
```

Item 21 Template : mettez le pack

Un deuxième cas d'usage dans les fonctions : **avec un opérateur** (*fold expressions*)

Pour aller plus loin : `(func(pack), ...)`

- Remarque : `(func(pack), ...)` n'est pas nécessairement un appel de fonction
 - peut-être une expression contenant un pack (écrite entre parenthèses)
 - permet alors d'exécuter une expression pour tous les arguments

Exo 21.3 Modifiez votre fonction d'affichage afin qu'elle insère des espaces entre chaque valeur. Vous n'utiliserez pas de récursion.

```
int main() {  
    print(1);  
    print(1, 'c'); // output: 1 c  
    print(1, "Hello World", 3.f); //output: 1 Hello World 3  
}
```

Item 21 Template : mettez le pack

Un troisième cas d'usage dans les fonctions : peut-on faire un **pack d'entier** ?

- oui : `template <int...ints>`

```
template <int...Ns>
void func() {
    std::cout << "int sequence size " << sizeof...(Ns) << std::endl;
    ((std::cout << Ns << ' '),...) << std::endl;
}

int main(){
    func<-1,0,2>();
    func<1,2,3>();
}
```

Item 21 Template : mettez le pack

Un troisième cas d'usage dans les fonctions : peut-on faire un **pack d'entier** ?

- oui : `template <int...ints>`
- on peut générer la séquence avec `std::integer_sequence<T,ints...>`
 - où `T` est le type d'entier

```
template <typename T, T...Ns>
void func(std::integer_sequence<T,Ns...> int_seq) {
    std::cout << "sequence size " << int_seq.size() << std::endl;
    ((std::cout << Ns << ' '),...) << std::endl;
}

int main(){
    func(std::integer_sequence<unsigned,0,1,2>{});
    func(std::make_integer_sequence<unsigned,3>{}); // <0,1,2>
}
```

Item 21 Template : mettez le pack

Un troisième cas d'usage dans les fonctions : peut-on faire un **pack d'entier** ?

- oui : `template <int...ints>`
- on peut générer la séquence avec `std::integer_sequence<T,ints...>`
 - où `T` est le type d'entier

Si le type d'entier = `size_t` on utilise `std::index_sequence<size_t...ints>`

```
template<typename Tuple, size_t...indexes>
void print_tuple(Tuple& tuple, std::index_sequence<indexes...>){
    ((std::cout << std::get<indexes>(tuple) << ' '), ...);
}
int main() {
    std::tuple<int, char, std::string> tuple {42, 'c', "hello"};
    print_tuple(tuple, std::make_index_sequence<3>{});
}
```

Item 21 Template : mettez le pack

Un troisième cas d'usage dans les fonctions : peut-on faire un **pack d'entier** ?

- oui : `template <int...ints>`
- mais : on ne peut pas les utiliser directement comme argument
 - il faut passer par une `std::integer_sequence<T,N...>`, où `T` un type d'entier

si le type d'entier = `size_t` on utilise `std::index_sequence<size_t...ints>`

Exo 21.4 (Avancé) Réalisez une méthode d'impression d'un tuple ne prenant en argument que le tuple (pas l' `index_sequence`) :

```
int main() {  
    std::tuple<int, char, std::string> tuple {42, 'c', "hello"};  
    print_tuple(tuple);  
}
```

Item 21 Template : mettez le pack

Cas d'usage pour les **classes** : une classe peut également prendre un pack

- ex d'un tuple pouvant contenir un nombre variable d'éléments de types différents

```
template <typename...Args>
class Tuple {
public:
    Tuple(Args...) {}
};

int main() {
    Tuple my_tuple{2,3.5,'c',"hello"};
}
```

Item 21 Template : mettez le pack

Cas d'usage pour les classes : une classe peut hériter d'un pack

- ex d'une classe héritant des classes données à la construction

```
template <typename...Args>
class Holder : public Args... {
public:
    Holder(const Args&...args): Args(args)... {}
};

struct Solver{};
struct Scheme {};

int main() {
    Holder<Solver,Scheme> holder(Solver{},Scheme{});
}
```